Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: L15+L16
Regno: *23BCE1344*	Name: *Aditya*

EX3

AES - CRYPTOGRAPHIC ALGORITHM

Index	
Code (Sender, Receiver)	
Output (Sender, Receiver)	

File name: Ex3_23BCE1344.pdf

Code: (--CyptoFunctions--)

Cypto_functions.py:

```
# crypto_functions.py

# AES CONSTANTS
SBOX = [
0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x
76,
0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0x
c0,
0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x
15,
0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x
75,
0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x
84,
0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0x
cf,
0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0x
a8,
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0x
d2,
0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x
73,
0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0x
db,
0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x
79,
0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x
08,
0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x
8a,
0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x
9e,
0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0x
df,
0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x
16
]

INV_SBOX = [0]*256
```

```python
for i, v in enumerate(SBOX):
    INV_SBOX[v] = i
RCON = [0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x1B,0x36]

# HELPER FUNCTIONS
def text_to_blocks(text):
    data = [ord(c) for c in text]
    while len(data) % 16 != 0:
        data.append(0)
    return [data[i:i+16] for i in range(0, len(data), 16)]
def block_to_hex(b):
    return ''.join(f"{x:02X}" for x in b)
def add_round_key(s, k):
    return [a ^ b for a, b in zip(s, k)]
def sub_bytes(s):
    return [SBOX[b] for b in s]
def inv_sub_bytes(s):
    return [INV_SBOX[b] for b in s]
def shift_rows(s):
    return [
        s[0],s[5],s[10],s[15],
        s[4],s[9],s[14],s[3],
        s[8],s[13],s[2],s[7],
        s[12],s[1],s[6],s[11]
    ]
def inv_shift_rows(s):
    return [
        s[0],s[13],s[10],s[7],
        s[4],s[1],s[14],s[11],
        s[8],s[5],s[2],s[15],
        s[12],s[9],s[6],s[3]
    ]
def gmul(a, b):
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi = a & 0x80
        a = (a << 1) & 0xFF
        if hi:
            a ^= 0x1B
        b >>= 1
    return p
```

```python
def mix_columns(s):
    o = []
    for i in range(0,16,4):
        a = s[i:i+4]
        o += [
            gmul(a[0],2)^gmul(a[1],3)^a[2]^a[3],
            a[0]^gmul(a[1],2)^gmul(a[2],3)^a[3],
            a[0]^a[1]^gmul(a[2],2)^gmul(a[3],3),
            gmul(a[0],3)^a[1]^a[2]^gmul(a[3],2)
        ]
    return o
def inv_mix_columns(s):
    o = []
    for i in range(0,16,4):
        a = s[i:i+4]
        o += [
            gmul(a[0],14)^gmul(a[1],11)^gmul(a[2],13)^gmul(a[3],9),
            gmul(a[0],9)^gmul(a[1],14)^gmul(a[2],11)^gmul(a[3],13),
            gmul(a[0],13)^gmul(a[1],9)^gmul(a[2],14)^gmul(a[3],11),
            gmul(a[0],11)^gmul(a[1],13)^gmul(a[2],9)^gmul(a[3],14)
        ]
    return o

# KEY EXPANSION
def key_expansion(key):
    key = [ord(c) for c in key.ljust(16)[:16]]
    keys = [key]
    for i in range(10):
        t = keys[-1][12:16]
        t = t[1:] + t[:1]
        t = [SBOX[b] for b in t]
        t[0] ^= RCON[i]
        new = []
        for j in range(16):
            if j < 4:
                new.append(keys[-1][j] ^ t[j])
            else:
                new.append(keys[-1][j] ^ new[j-4])
        keys.append(new)
    return keys

# AES ENCRYPTION (ECB)
def aes_encrypt_ecb(text, key, verbose=True):
```

```python
    blocks = text_to_blocks(text)
    keys = key_expansion(key)
    out = ""
    if verbose:
        print(f"\nECB MODE: {len(blocks)} block(s)")
    for idx, b in enumerate(blocks,1):
        s = add_round_key(b, keys[0])
        if verbose:
            print(f"\n--- Encrypting Block {idx} ---")
            print("Round 0 :", block_to_hex(s))
        for r in range(1,10):
            s = sub_bytes(s)
            s = shift_rows(s)
            s = mix_columns(s)
            s = add_round_key(s, keys[r])
            if verbose:
                print(f"Round {r} :", block_to_hex(s))
        s = sub_bytes(s)
        s = shift_rows(s)
        s = add_round_key(s, keys[10])
        if verbose:
            print("Round 10:", block_to_hex(s))
        out += block_to_hex(s)
    return out

# AES DECRYPTION (ECB)
def aes_decrypt_ecb(hex_ct, key, verbose=True):
    data = bytes.fromhex(hex_ct)
    blocks = [list(data[i:i+16]) for i in range(0,len(data),16)]
    keys = key_expansion(key)
    text = ""
    if verbose:
        print(f"\nECB MODE: {len(blocks)} block(s)")
    for idx, s in enumerate(blocks,1):
        if verbose:
            print(f"\n--- Decrypting Block {idx} ---")
            print("Initial:", block_to_hex(s))
        s = add_round_key(s, keys[10])
        for r in range(9,0,-1):
            s = inv_shift_rows(s)
            s = inv_sub_bytes(s)
            s = add_round_key(s, keys[r])
            s = inv_mix_columns(s)
```

```python
            if verbose:
                print(f"Round {r} :", block_to_hex(s))
        s = inv_shift_rows(s)
        s = inv_sub_bytes(s)
        s = add_round_key(s, keys[0])
        if verbose:
            print("Plaintext Block:", block_to_hex(s))
        text += ''.join(chr(b) for b in s)
    return text.rstrip('\x00')

# DEMO RUN
def main():
    pt = "cryptography"
    key = "Security"
    print("Plaintext:", pt)
    print("Key       :", key)
    ct = aes_encrypt_ecb(pt, key)
    print("\nCiphertext (HEX):", ct)
    dec = aes_decrypt_ecb(ct, key)
    print("\nDecrypted Text:", dec)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
(lab) ak@1125:~/Desktop/Data/CryptoLab$ python crypto_functions.py
Plaintext: cryptography
Key      : Security

ECB MODE: 1 block(s)

--- Encrypting Block 1 ---
Round 0 : 30171A050606130B4150485920202020
Round 1 : B949D98660DB5438E1BB96465FBB5B79
Round 2 : 57102BEE79991AC1D17D6FA8082AE434
Round 3 : F5FBD6256CD57D48AC982A83922EC653
Round 4 : EC17A7A6971F0583D59C0011503515CC
Round 5 : 9D4E9B87102E274813CE0CD28CF77981
Round 6 : 845D9E262EABA241868B68968AE83E07
Round 7 : 75C6F3920B9CCCFB5F52495A21B5C1ED
Round 8 : 0604493C3A2734865108318AC20E0AA0
Round 9 : 8C86B82CA6B1D351F3E86B26E312C20B
Round 10: 634C8A43420C5B49B823B9BF663014EB

Ciphertext (HEX): 634C8A43420C5B49B823B9BF663014EB

ECB MODE: 1 block(s)

--- Decrypting Block 1 ---
Initial: 634C8A43420C5B49B823B9BF663014EB
Round 9 : 6FCCC7E0803067EBD1AB3B4425F2187E
Round 8 : 9DDE3B552B00784FCFD50D0FFDB44BBE
Round 7 : 5F6245C5313DB2F7449B0B837E4C3A90
Round 6 : 5E31FE0CCA8BB6177D681452642FCCB5
Round 5 : CEC0634B88DE592403965CEC53F06B82
Round 4 : E603E5ED5046B43F9131F6524F0FFFEC
Round 3 : 5BEEA818B6FF69283EE5F17830CAA2C2
Round 2 : 56B990B6D0EA3944F8EA3507CF3B205A
Round 1 : 046F52B76F53B76B83B7A22BB7F07DCB
Plaintext Block: 63727970746F67726170687900000000

Decrypted Text: cryptography
(lab) ak@1125:~/Desktop/Data/CryptoLab$
```

Sender (server.py):

```
import socket
import crypto_functions as cf

KEY = "Security"

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 12345))
server.listen(1)
print("Server ready (AES Encryption)")
conn, addr = server.accept()
print("Connected:", addr)
while True:
   pt = input("\nEnter Plaintext (or exit): ")
   if pt.lower() == "exit":
       conn.send(b"exit")
       break

   ct = cf.aes_encrypt_ecb(pt, KEY, verbose=True)
   conn.send(ct.encode())
conn.close()
server.close()
```

Receiver (client.py):

```
import socket
import crypto_functions as cf
KEY = "Security"
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 12345))
print("Connected to server (AES Decryption)")
while True:
   ct = client.recv(4096).decode()
   if ct == "exit":
       break

   print("\nReceived Ciphertext:", ct)
   pt = cf.aes_decrypt_ecb(ct, KEY, verbose=True)
   print("\nDecrypted Text:", pt)

client.close()
```

Output:

```
(lab) ak@1125:~/Desktop/Data/CryptoLab$ python server.py
Server ready (AES Encryption)
Connected: ('127.0.0.1', 47272)

Enter Plaintext (or exit): cryptography

ECB MODE: 1 block(s)

--- Encrypting Block 1 ---
Round 0 : 30171A050606130B4150485920202020
Round 1 : B949D98660DB5438E1BB96465FBB5B79
Round 2 : 57102BEE79991AC1D17D6FA8082AE434
Round 3 : F5FBD6256CD57D48AC982A83922EC653
Round 4 : EC17A7A6971F0583D59C0011503515CC
Round 5 : 9D4E9B87102E274813CE0CD28CF77981
Round 6 : 845D9E262EABA241868B68968AE83E07
Round 7 : 75C6F3920B9CCCFB5F52495A21B5C1ED
Round 8 : 0604493C3A2734865108318AC20E0AA0
Round 9 : 8C86B82CA6B1D351F3E86B26E312C20B
Round 10: 634C8A43420C5B49B823B9BF663014EB

Enter Plaintext (or exit): Cryptography

ECB MODE: 1 block(s)

--- Encrypting Block 1 ---
Round 0 : 10171A050606130B4150485920202020
Round 1 : 3E8717CF60DB5438E1BB96465FBB5B79
Round 2 : 84F4CFD9B75753461429FE6D7C72C818
Round 3 : 9F0E85FE20FB068892A2DB4EFCCB14EE
Round 4 : 1B137A200C569590F72B931011F12B7F
Round 5 : EA8E5EC3B6769B6896D6C8DDB0AC7FB5
Round 6 : F97512E0E740D8BD0C4CDD216100BC9F
Round 7 : C55F6F10A765F2E018A7845C461D8015
Round 8 : B6A23D9E00A9FAE74D925A3F2CF64E18
Round 9 : DB5FA3B35294E4837BE7E33E3699291C
Round 10: BEA6E4F46603DB559404DF8272BB1BAE

Enter Plaintext (or exit):

ECB MODE: 0 block(s)

Enter Plaintext (or exit):
```

```
(lab) ak@1125:~/Desktop/Data/CryptoLab$ python client.py
Connected to server (AES Decryption)

Received Ciphertext: 634C8A43420C5B49B823B9BF663014EB

ECB MODE: 1 block(s)

--- Decrypting Block 1 ---
Initial: 634C8A43420C5B49B823B9BF663014EB
Round 9 : 6FCCC7E0803067EBD1AB3B4425F2187E
Round 8 : 9DDE3B552B00784FCFD50D0FFDB44BBE
Round 7 : 5F6245C5313DB2F7449B0B837E4C3A90
Round 6 : 5E31FE0CCA8BB6177D681452642FCCB5
Round 5 : CEC0634B88DE592403965CEC53F06B82
Round 4 : E603E5ED5046B43F9131F6524F0FFFEC
Round 3 : 5BEEA818B6FF69283EE5F17830CAA2C2
Round 2 : 56B990B6D0EA3944F8EA3507CF3B205A
Round 1 : 046F52B76F53B76B83B7A22BB7F07DCB
Plaintext Block: 63727970746F67726170687900000000

Decrypted Text: cryptography

Received Ciphertext: BEA6E4F46603DB559404DF8272BB1BAE

ECB MODE: 1 block(s)

--- Decrypting Block 1 ---
Initial: BEA6E4F46603DB559404DF8272BB1BAE
Round 9 : 4ED3BEAD634F2F0BE3422794713A2D75
Round 8 : A64D5F595C5CCDCAADA4A8E15ACF894A
Round 7 : 9909C1DB942965E1FE63C97AEF9D61FD
Round 6 : 8738E8D54EF6D22E90915845E71914C1
Round 5 : AFB1DCD2FEF1F1B768A1DA60827D2ACA
Round 4 : DB0FB928B73AFABB4F1F97C4B0AB6F2F
Round 3 : 5F5BBBADA9A5E835FA408A5A10BFED3C
Round 2 : B2B990B6D0EA398AF8EAF007CF17205A
Round 1 : CA6F52B76F53B76B83B7A22BB7F07DCB
Plaintext Block: 43727970746F67726170687900000000

Decrypted Text: Cryptography
```